

ArchiMate Examples

By Eero Hosiaisluomoa

Snapshot from the blog (2018-09-01)

<http://www.hosiaisluomoa.fi/blog/>

Table of Contents

1.	Introduction	3
2.	ArchiMate Example Views	4
	Framework View	4
3.	Motivation Views	5
	Motivation View	5
	Mission-Values-Vision View	5
	Strategic Value Map View	6
	Stakeholder Analysis View	7
	Stakeholder View	7
	Principles View	8
	Risk & Security View	8
	SWOT Analysis View	8
4.	Strategy Views	9
	Strategy View	9
	Business Model View	10
	Business Motivation Model (BMM) View	11
	Requirements View	11
	Strategy to Capability View	12
	Capability Map View	13
	Capability Planning View	13
	Capability Realization View	14
5.	Business Views	15
	Business Services Map View	15
	Business Process Map View	15
	Business Process Co-Operation View	16
	Business Actors Map View	16
	Business Actor Co-Operation View	16
	Business Process View	17
	Business Process View With Swimlanes As Roles of a Process - A Layered Approach	17
	Customer Journey Map View	18
	Service Blueprint View	19
	User Story View	20
	Cloud-Service Models View	20
	Information View	21
	Conceptual Data Model View	21
	"Service" Concept	21
6.	Application Views	22
	Application Services Map View	22
	Applications Map View	22
	Application Co-Operation View (Data flows)	22
	Application Integration View (Dynamic relationships)	22
	Application Integration View	23
	Application Structure View	24
	Application Functions View	24
	Layered View	25
7.	Technology Views	26
	Infrastructure View	26
8.	Implementation and Migration Layer / Transformation Architecture Layer Views	27
	Generic View	27
	Kanban View	27

1. Introduction

This is a snapshot from <http://www.hosiaisluomoa.fi/blog/archimate-examples/> (taken 1st Sep 2018).

ArchiMate (3.x) example views introduced here are organized into a layered framework according to ArchiMate standard (see ArchiMate version 3.0.1 [here](#)). These example views demonstrate how ArchiMate concepts can be used. Some of the examples can be used as design patterns.

2. ArchiMate Example Views

Framework View

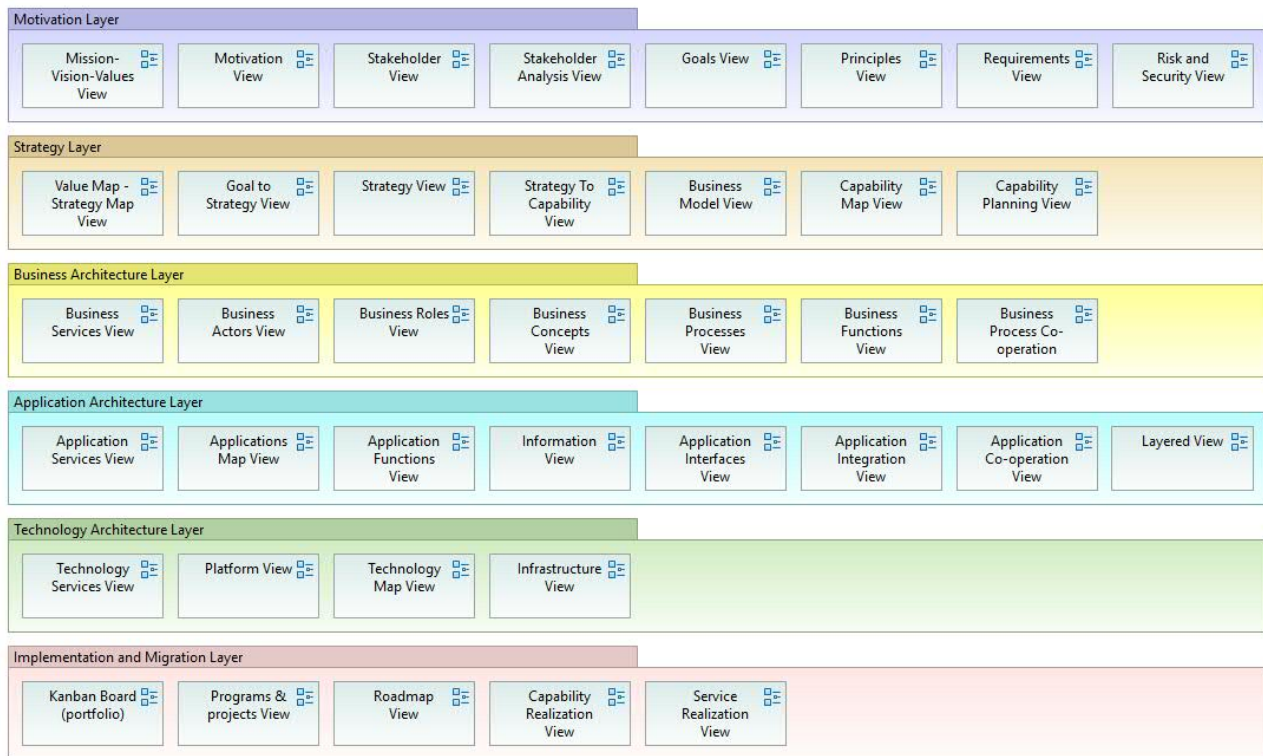


Figure 1: Framework View.

This view represents the framework that structures all the development aspects and related diagrams. The view can be modified according to what is appropriate in the case. As such, this view can be used for navigation between the diagrams. This version of the view is applied from ArchiMate (3) framework. Motivation is introduced as a "Layer" instead of an "Aspect".

3. Motivation Views

Motivation View

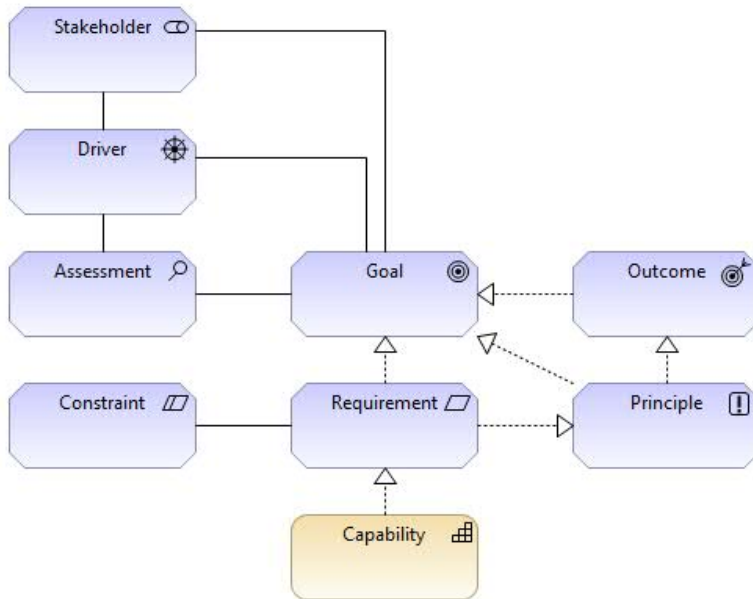


Figure 2: Motivation View.

This view can be used to analysis of the motivations, or reasons, that guide the design or change of an organization and its enterprise architecture. These motivational analysis views are the starting points for all the change activities or business transformations in an organization. This view represents the vision of the development endeavour - whether the scale and scope comprises the whole organization or just part of it (e.g. line of business) or single program or project (solution level).

Motivational elements are based on Business Motivation Model (BMM) [[specification](#) v.1.3, 2015, OMG].

Mission-Values-Vision View

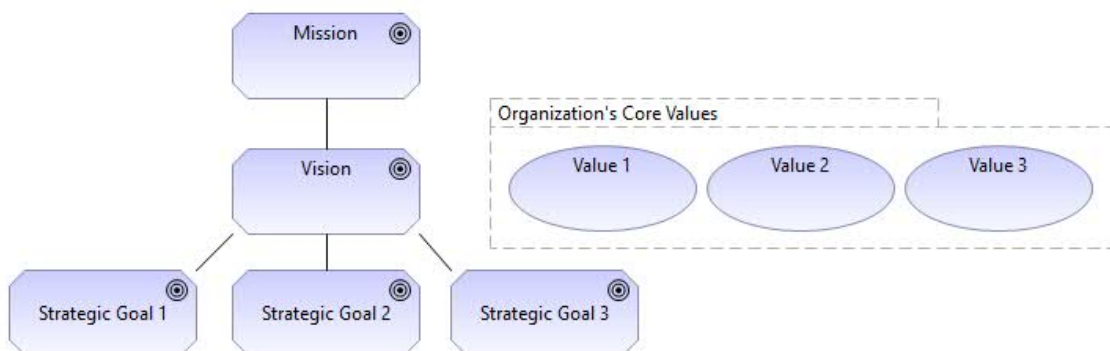


Figure 3: Mission-Vision-Values View.

This view can be used to represent the organization's mission, vision and core values. Mission expresses e.g. "What is the organization's purpose, what is it actually doing or intends to do, what is the primary reason for its existence?" Vision is the future state towards which the organization intends to evolve. Core values are what support the vision, shape the culture, and reflect what the organization values. Strategic goals need to be accomplished in order to achieve the vision of the organisation.

Reference: Aldea, A. – Iacob, M.-E. – Hillegersberg, J. - Quartel, D. – Franken, H. (2015) Modelling strategy with ArchiMate.

Strategic Value Map View

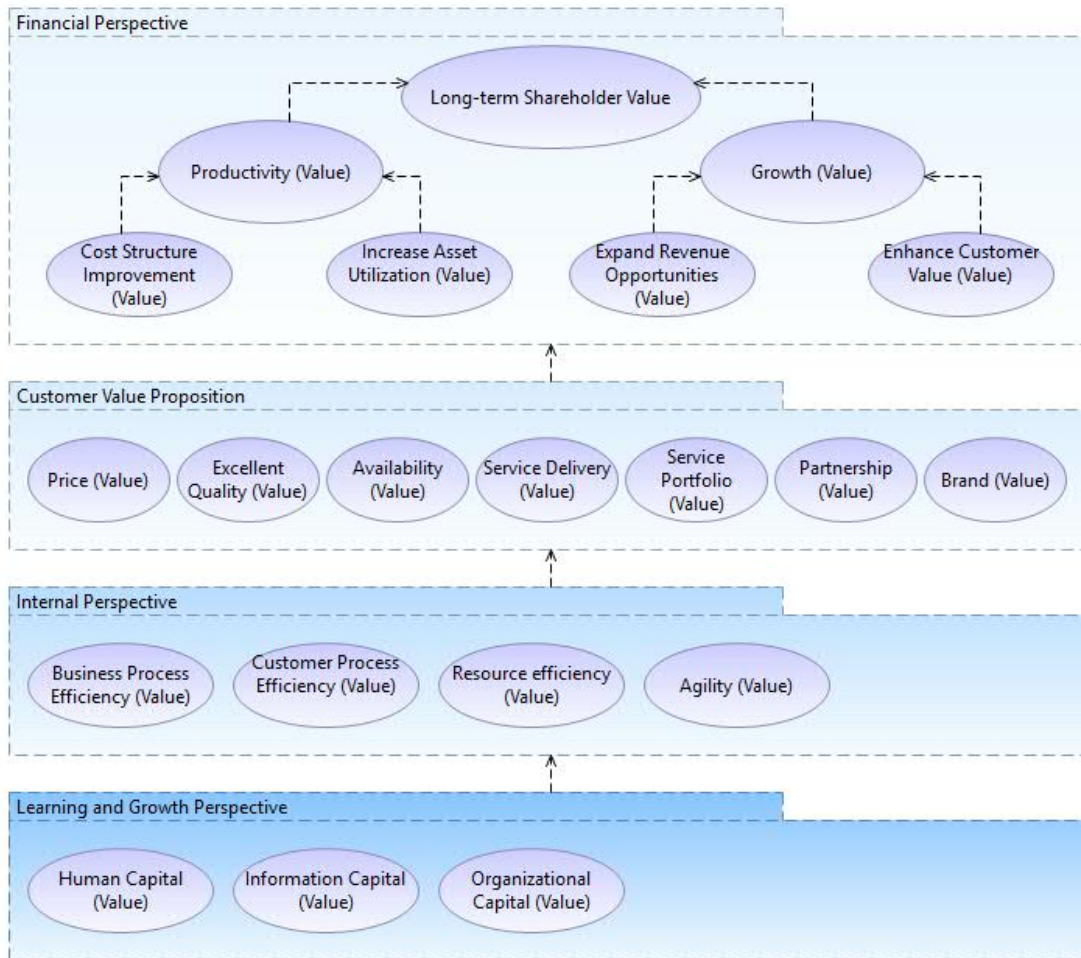


Figure 4: Value Map - Strategy Map View.

This view can be used for visualization of the strategies of the organization. This view contains strategic value-elements, from which all the development activities have to be derived - directly or indirectly. By visualizing the strategic values, it can be possible to trace all the other elements that are involved with the actual strategy execution. With this view, the strategy can be made available: visualized, communicated and linked to the reality.

Stakeholder Analysis View

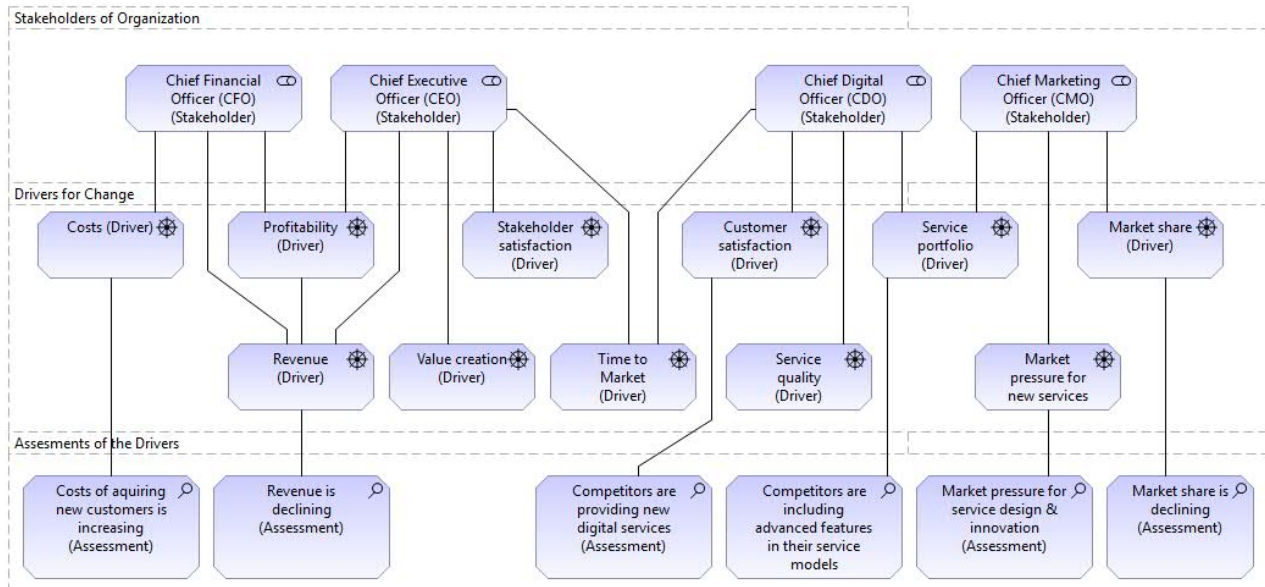


Figure 5 Stakeholder Analysis View.

This view can be used for stakeholder analysis for business development purposes: what are the drivers for change. First identify the relevant stakeholders and then the drivers for change that are in the interests of them. The "Assesment" concepts can be used for detailed analysis of drivers, e.g. according to SWOT (Strengths, Weaknesses, Opportunities, Threats) method. As typical, distinct stakeholder view diagrams can be created from different viewpoints. Another reason for splitting large diagrams into smaller, is to keep the diagrams compact and readable - for the sake of simplicity.

Stakeholder View

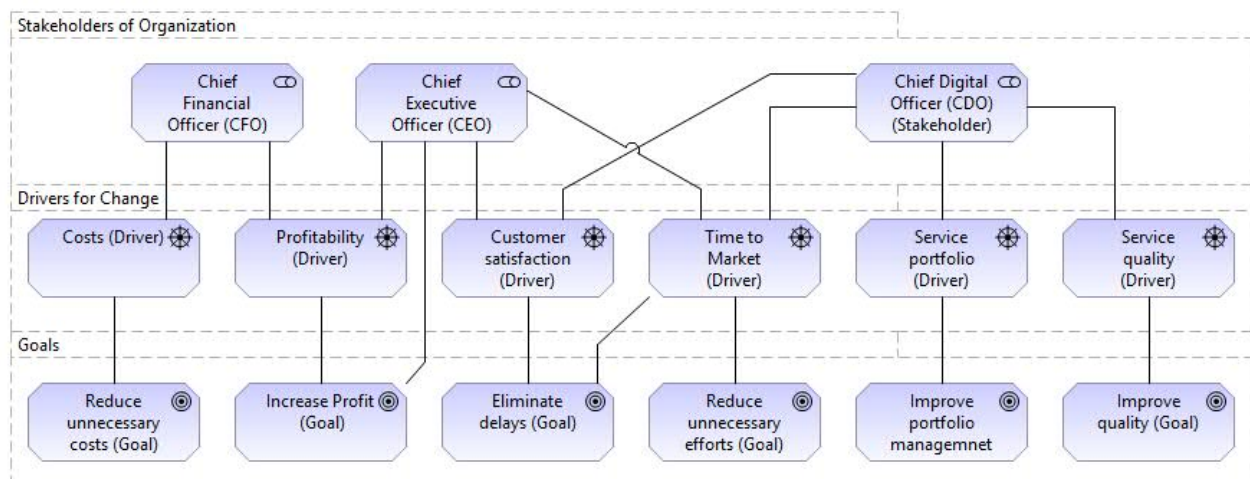


Figure 6: Stakeholder View.

This view can be used for linking stakeholder's drivers to business goals. Goals are the key elements of development in an organization. All the subsequent elements should be traced back to these primary reasons for all the change activities.

Principles View

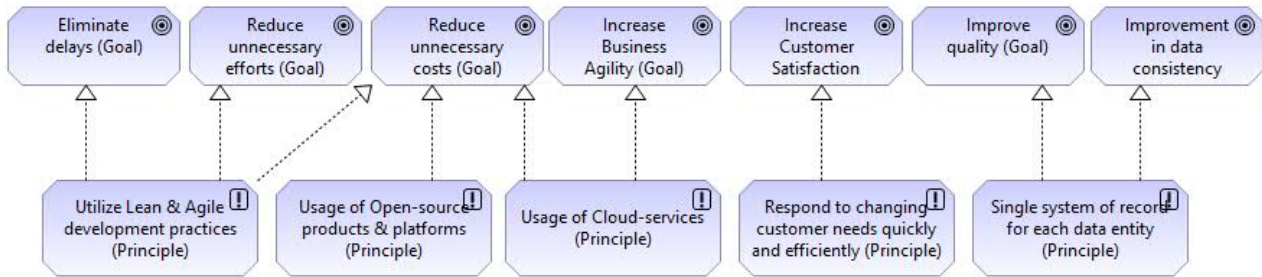


Figure 7: Principles View.

Risk & Security View

Mapping of Risk and Security Concepts to the ArchiMate.

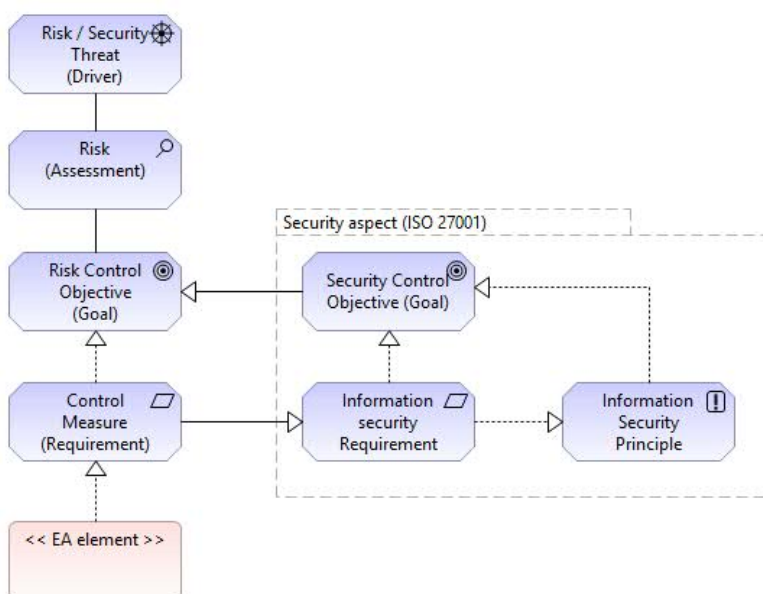


Figure 8: Risk and Security View.

SWOT Analysis View

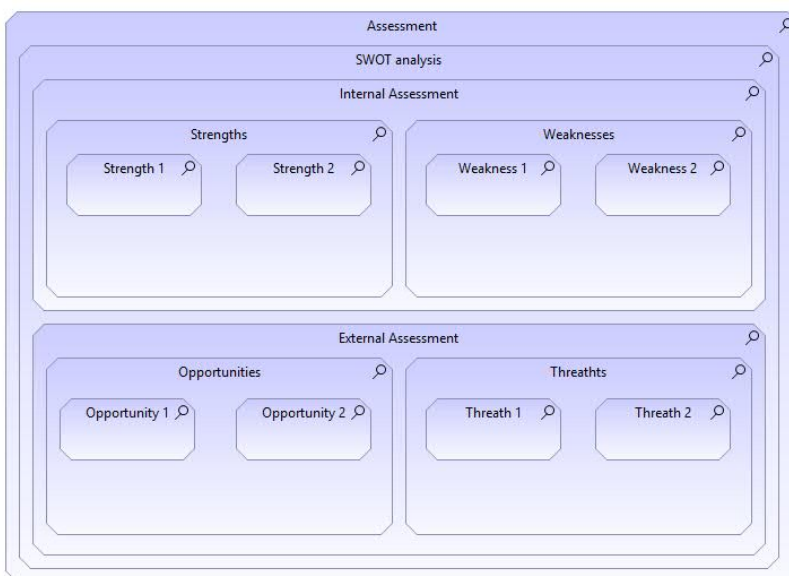


Figure 9: SWOT analysis View.

4. Strategy Views

Strategy View

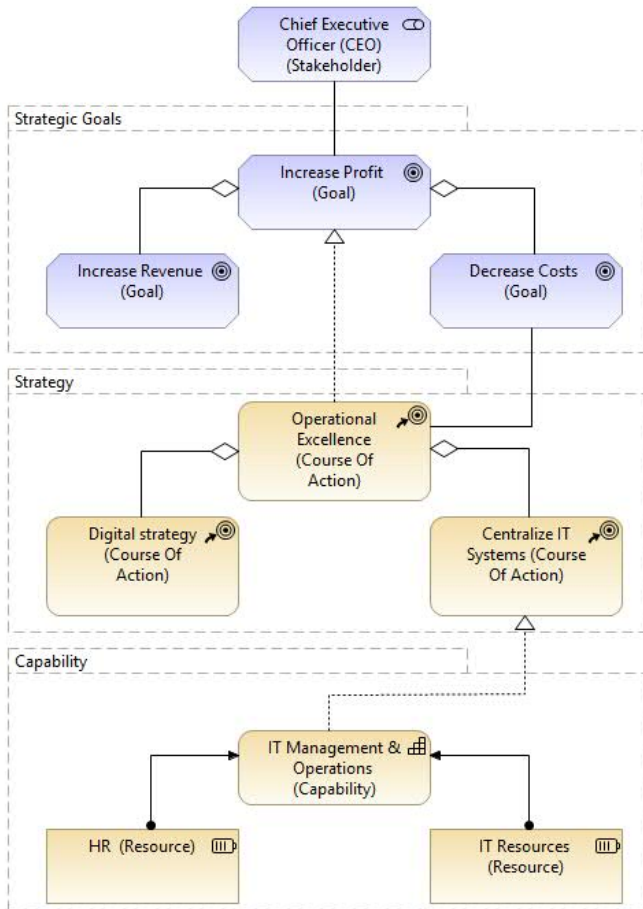


Figure 10: Strategy View.

ArchiMate version 3 now supports business strategy related concepts such as "Course of Action", "Capability" and "Resource", which can be used for modeling business strategies of the organization. The value and importance of this view is in the way that the goals of the organization can be linked to strategies, and how they can be linked into enterprise architecture via capabilities. This view can be used to apply the "Goal-Based Strategic Model" (Azevedo et al. 2015), in which goals constitute a hierarchy, so that higher-level goals can be decomposed into lower-level goals.

Business Model View

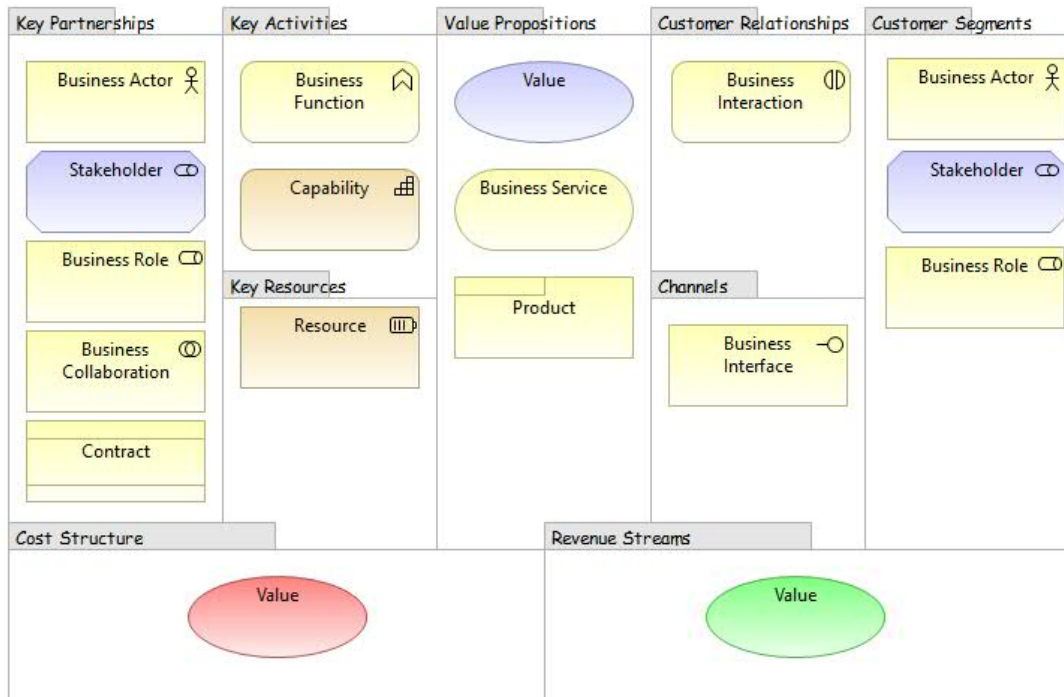


Figure 11: Business Model View.

This is the basic form of the [Business Model Canvas](#) (BMC) by A. Osterwalder, but it can be varied according to what is appropriate. There are also versioned approaches such as "Service Model Canvas" or "Lean Canvas". A BMC can be used e.g. for business model design and innovation.

Modeling a BMC with ArchiMate "facilitates tracing of requirements from business demands down to the design specifications. This helps discovering the effects of business model changes on architectural design." [L.O. Meertens et al.]

Holistic development includes built-in architecture-support for strategy and business model analysis. This enables business analysts and developers observe e.g. how well business model supports the strategy and how business model fit into the organization, and vice versa.

If the BMC is modeled within a modeling tool, an advantage of this approach is that all the elements of the BMC can be used in other views of the same model repository. And when *pivoting* the business model, all the changes are immediately visible. Business modelers can create new elements such as services, or utilize all the existing elements in the repository such as organization units or resources.

Business Motivation Model (BMM) View

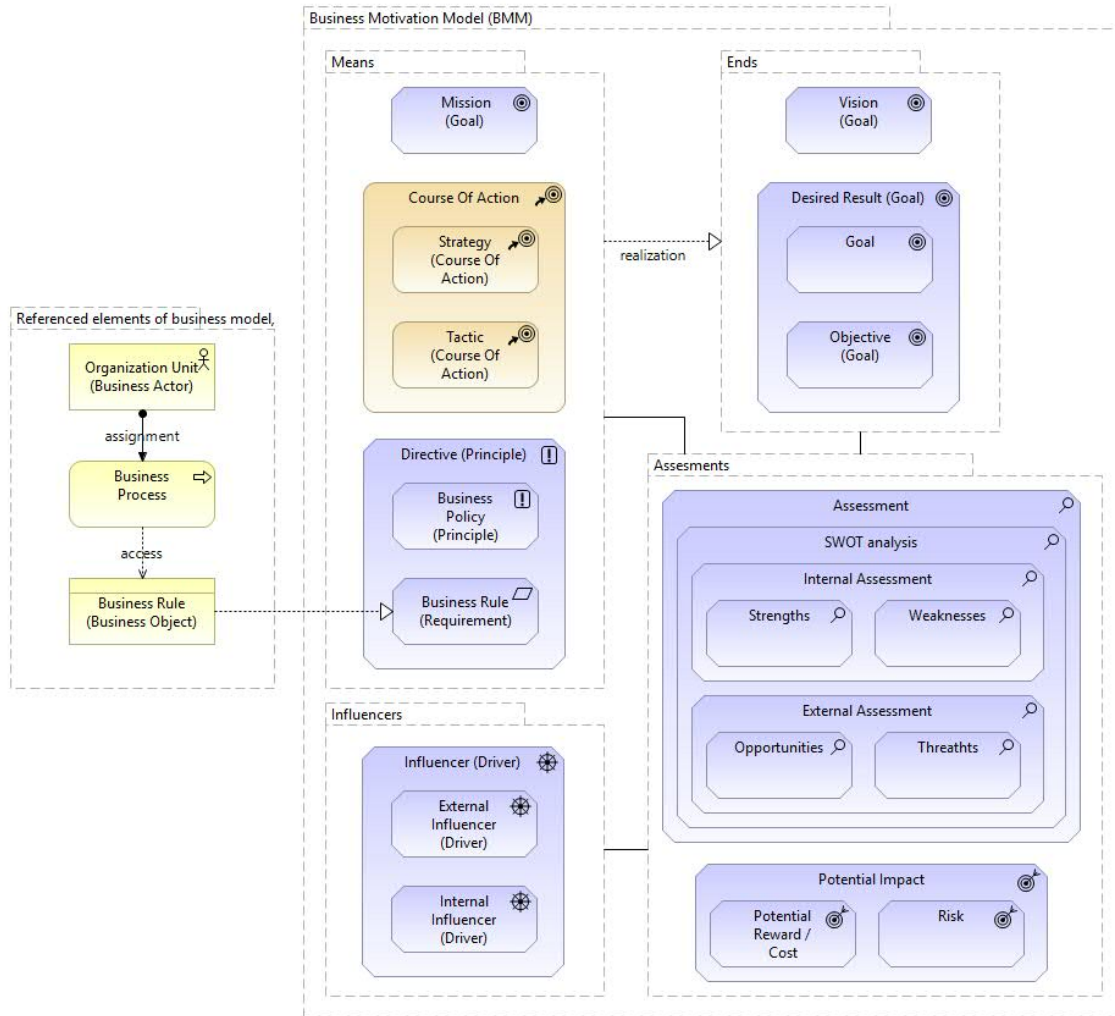


Figure 12: Business Motivation Model (BMM) View.

Requirements View

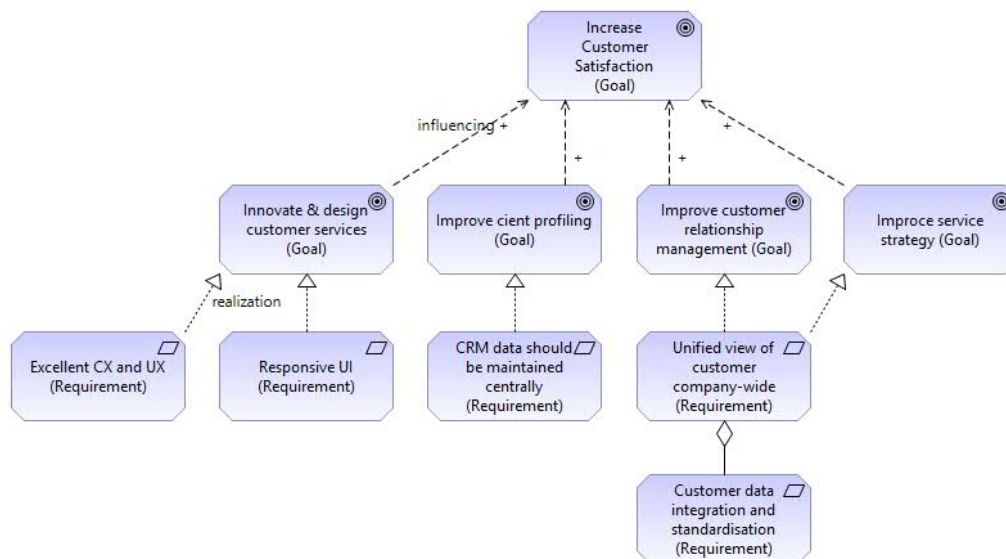


Figure 13: Requirements View.

This view can be used to gathering requirements based on the strategic goals. This is linking the strategies to implementations: it is possible to trace strategies to execution.

Strategy to Capability View

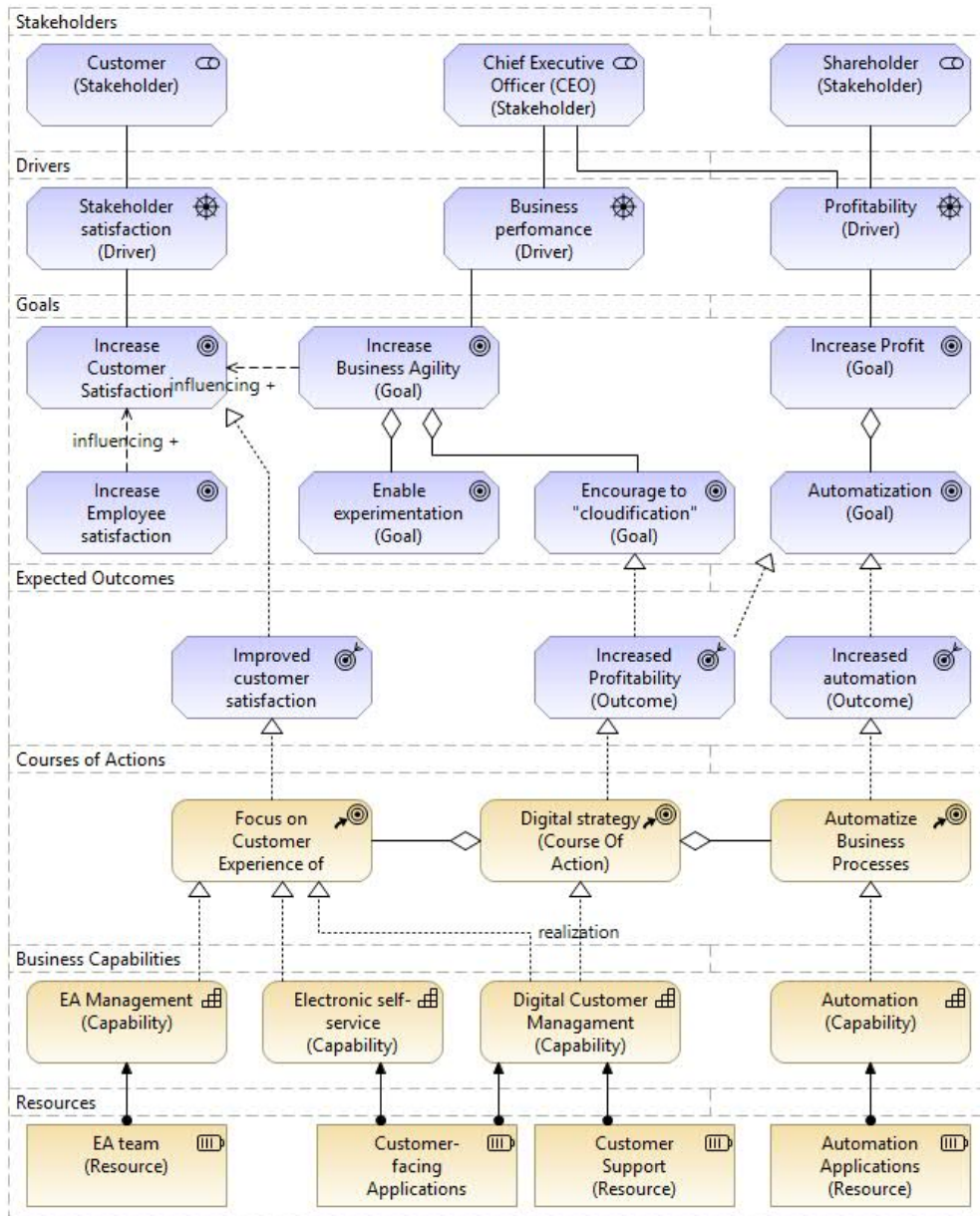


Figure 14: Strategy To Capability View.

This view can be used for *Capability-Based Planning (CBP)* purposes, together with other ArchiMate concepts such as "Driver" and "Goal" as shown in the diagram below. This view can be used to support *Strategy Planning* (and -Execution) purposes. As such, this kind of views can be used in Strategy-to-Capability phase, which can be included in the "Strategy-to-Portfolio" in IT4IT.

Capability Map View

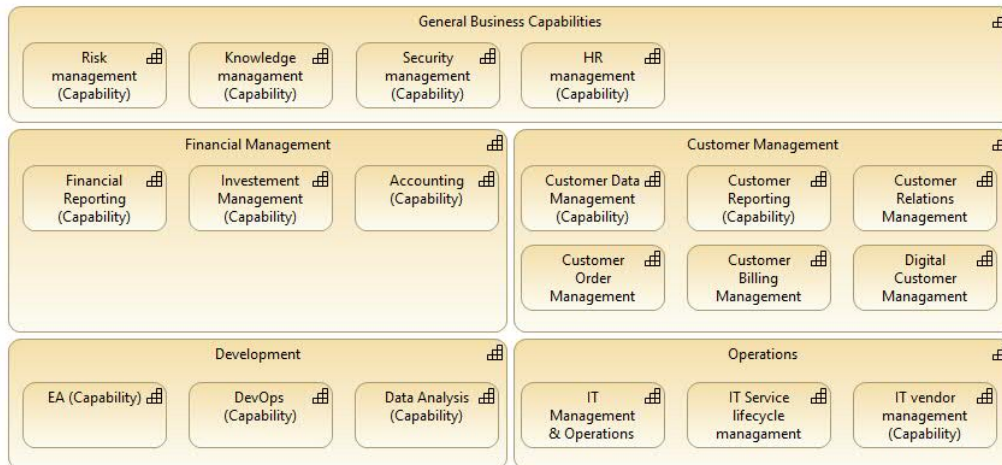


Figure 15: Capability Map View.

This view can be used for giving an overview of organizations capabilities: what the organization does or can do.

Capability Planning View

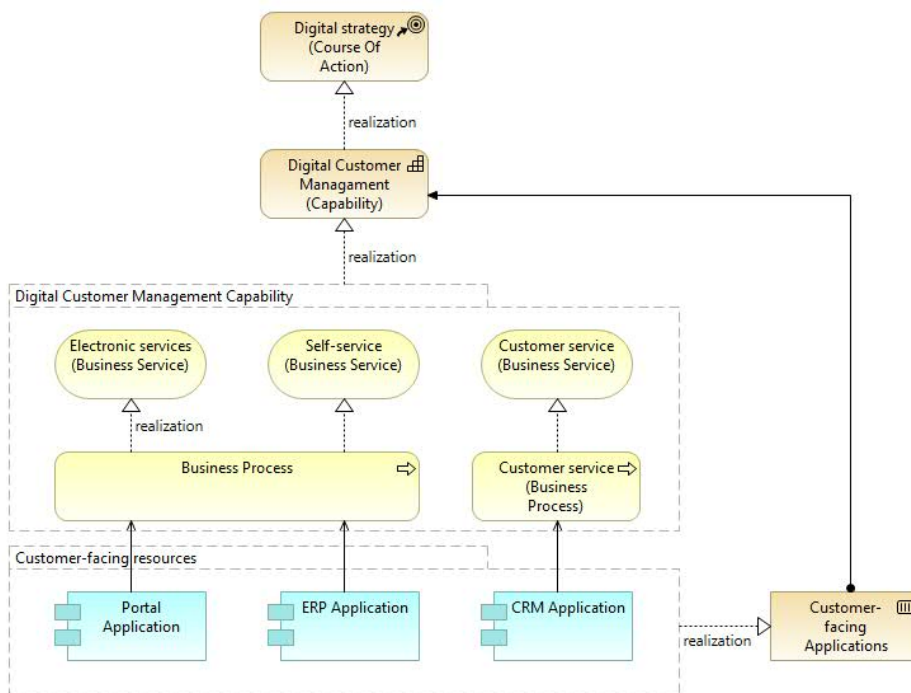


Figure 16: Capability Planning View.

This view can be used for e.g. *Capability-Based Planning (CBP)* purposes, which is "the Link between Strategy and Enterprise Architecture". This view can be used for e.g. mapping strategies to required capabilities, and mapping capabilities to resources and other building blocks.

Capability Realization View

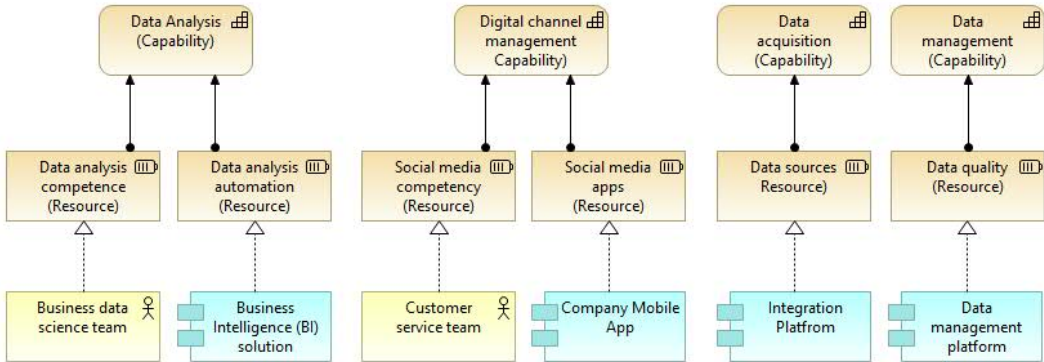


Figure 17: Capability Realization View.

5. Business Views

Business Architecture Layer Views.

In each layer there are several "maps" of elements that are managed within the EA-tool, such as Business Services Map, Process Map etc. After identified and introduced maps, those elements may be used in other diagrams (such as layered views). The purpose of the maps is to manage catalogs of "EA assets" as "portfolios" (analogous to portfolios of ideas, services and projects etc.). EA-tools typically provide other features for each element, e.g. properties or attributes. Those can be used to provide additional information per each element. This kind of extra information can also be used in different kinds of analysis purposes.

There can be several maps on each layer e.g. as follows:

- In Business Layer: Business Service, Business Actors, Business Processes
- In Application Layer: Application Services, Applications
- In Technology Layer: Technology Services, Platforms, Technologies etc.

Some example business layer maps are introduced here.

Business Services Map View



Figure 18: Business Services View.

This view gives an overview of the business services of the organization. This kind of view can be used as "Service Catalog" or "Service Portfolio" management purposes. It is important to identify what are the business services the organization is providing to its customers. In addition, a business service is a starting point for modelling all the underlying organizational processes and structures. As such, business services are the most important elements of the enterprise architecture.

Business Process Map View

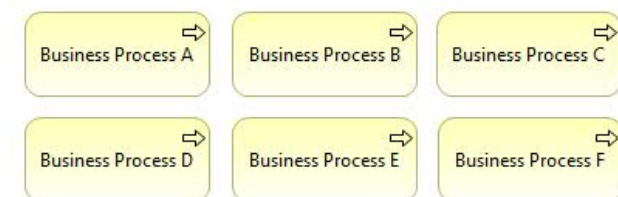


Figure 19: Business Processes View.

This view can be used as "Process Map" which gives an overview of the business processes of the organization.

Business Process Co-Operation View

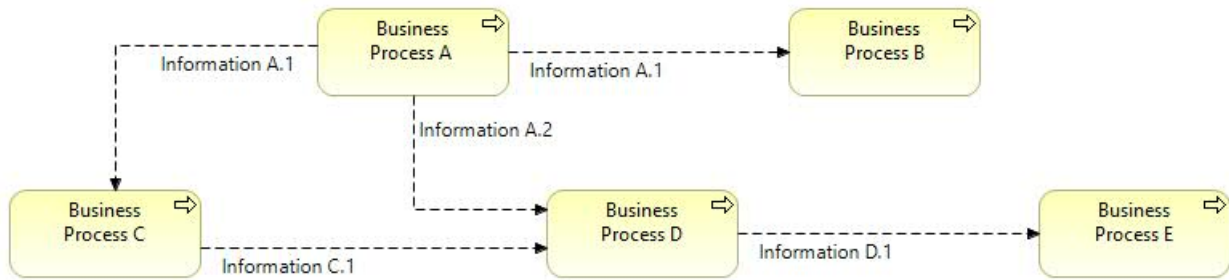


Figure 20: Business Process Co-operation View.

Business Actors Map View

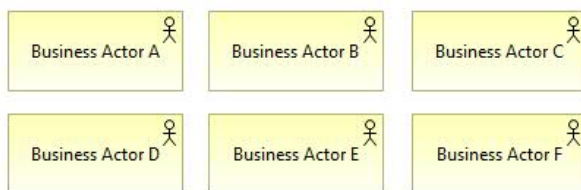


Figure 21: Business Actors Map View.

Business Actors can be a) internal or b) external. Internal business actors are e.g. organization units, and external business actors are e.g. customers, business partners, or other stakeholder groups that co-operate with the organization (such as public sector organizations or other governance authorities).

Business Actor Co-Operation View

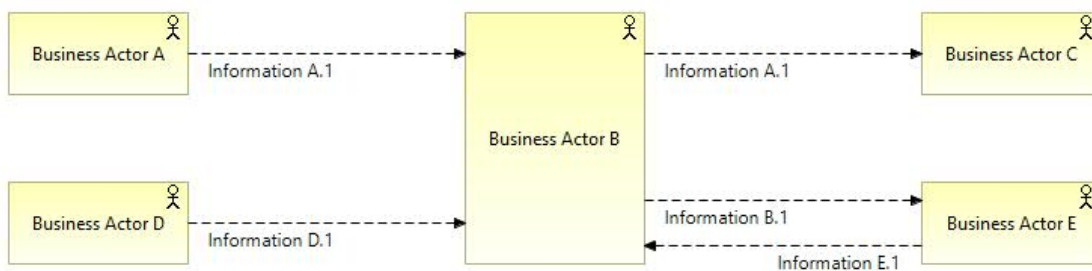


Figure 22: Business Actor Co-Operation View.

a) Intra-Enterprise View: Business actor co-operation view that depicts how internal business actors co-operate, how they switch information.

b) Inter-Enterprise View: The Ecosystem viewpoint that depicts the operational environment in which an organization operates is a network of organizations and business partners, what are the concerned interactions and collaborations. There are suppliers, sub-contractors and other b2b partners, customers etc.

Business Process View

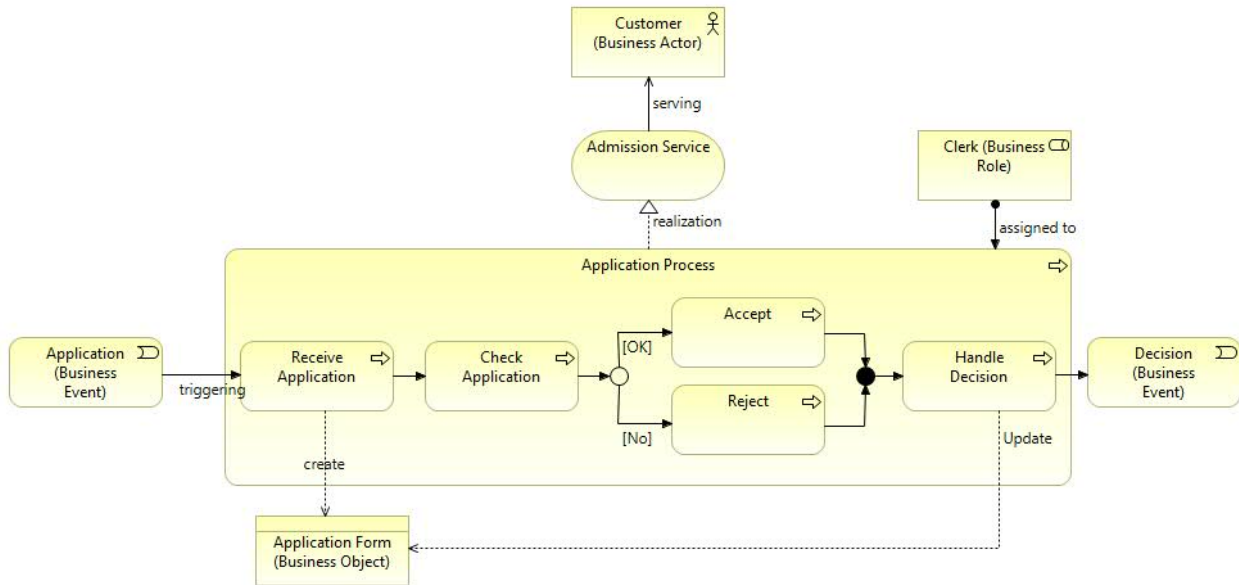


Figure 23: Business Process View.

This business process view provides a "high-level structure and composition of a business process (or several processes), the **services** that are offered, the assigned **roles of actors**, and the **information** used by the business process" [ArchiMate 2.1 specification]. This process diagram contains "Junction" -elements to model "fork" and "join" in the process flow.

Business Process View With Swimlanes As Roles of a Process - A Layered Approach

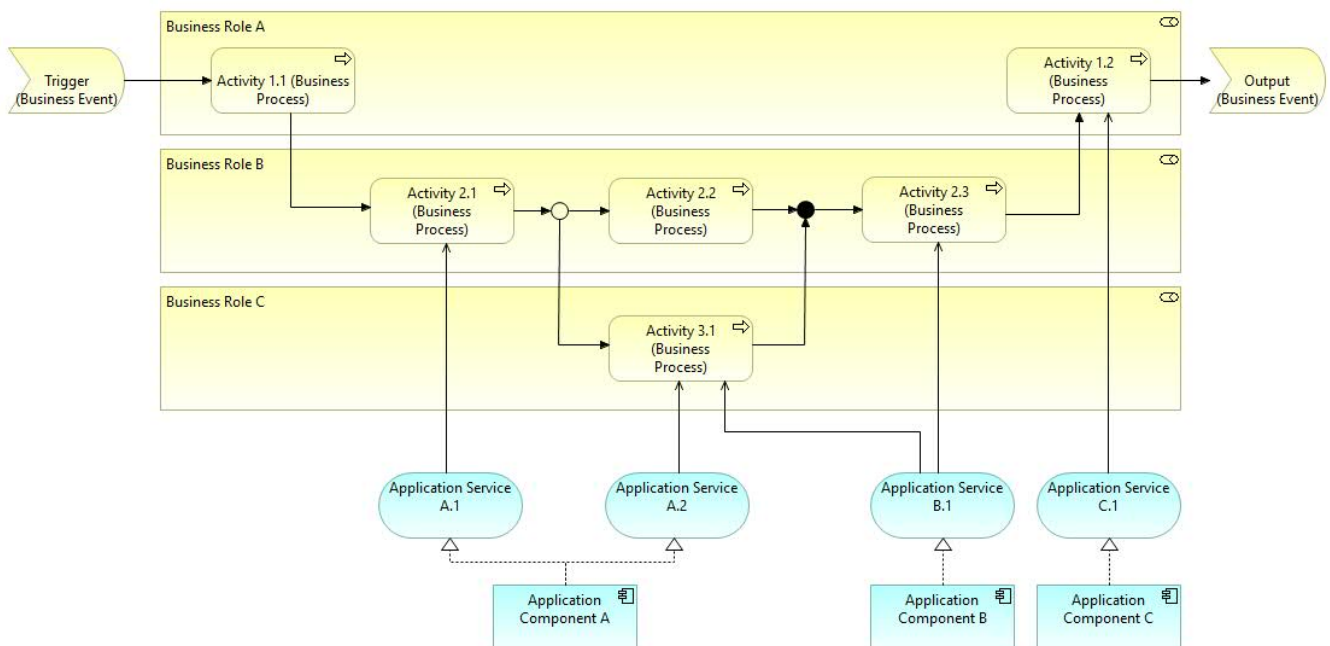


Figure 24: Business Process View, With Swimlanes As Roles of a Process - A Layered Approach.

Customer Journey Map View

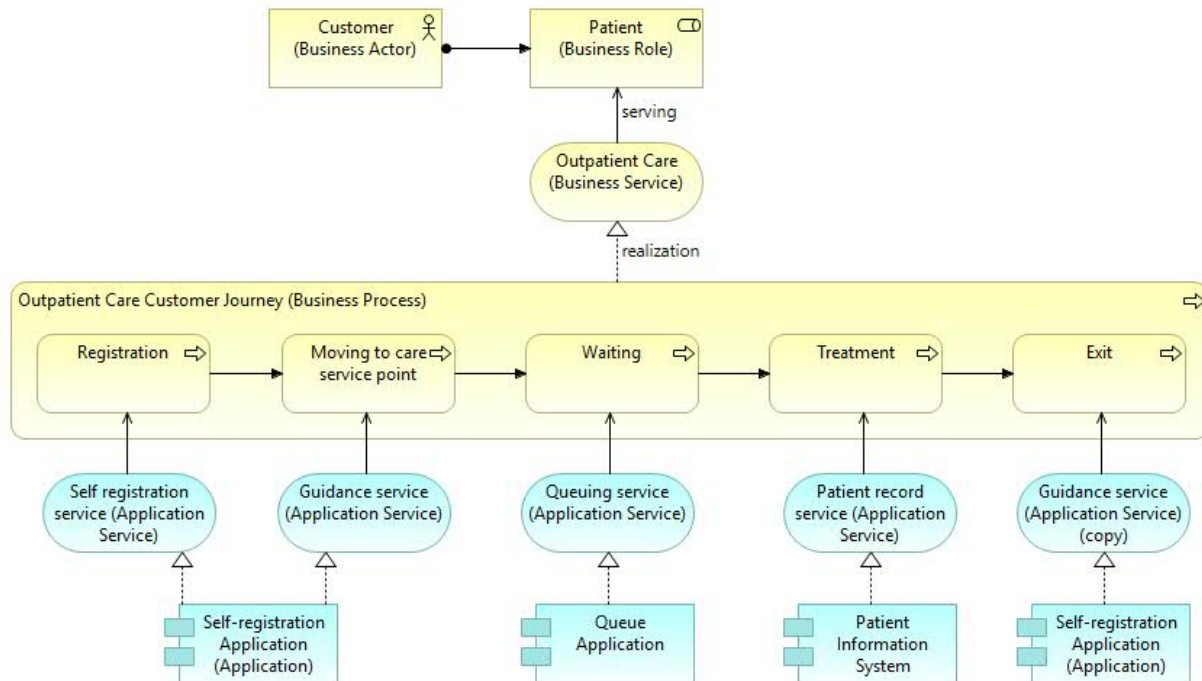


Figure 25: Customer Journey View.

This customer-centric viewpoint is focusing on customer experience. This "service design" related approach is concentrating the "outside-in" development of the service that is to be designed. This highlights the services and products as essential aspects that produces value to customers - and indirectly to the organization itself. A customer journey path can be used to visualization of a customer value stream, which spans over several application services and applications.

Service Blueprint View

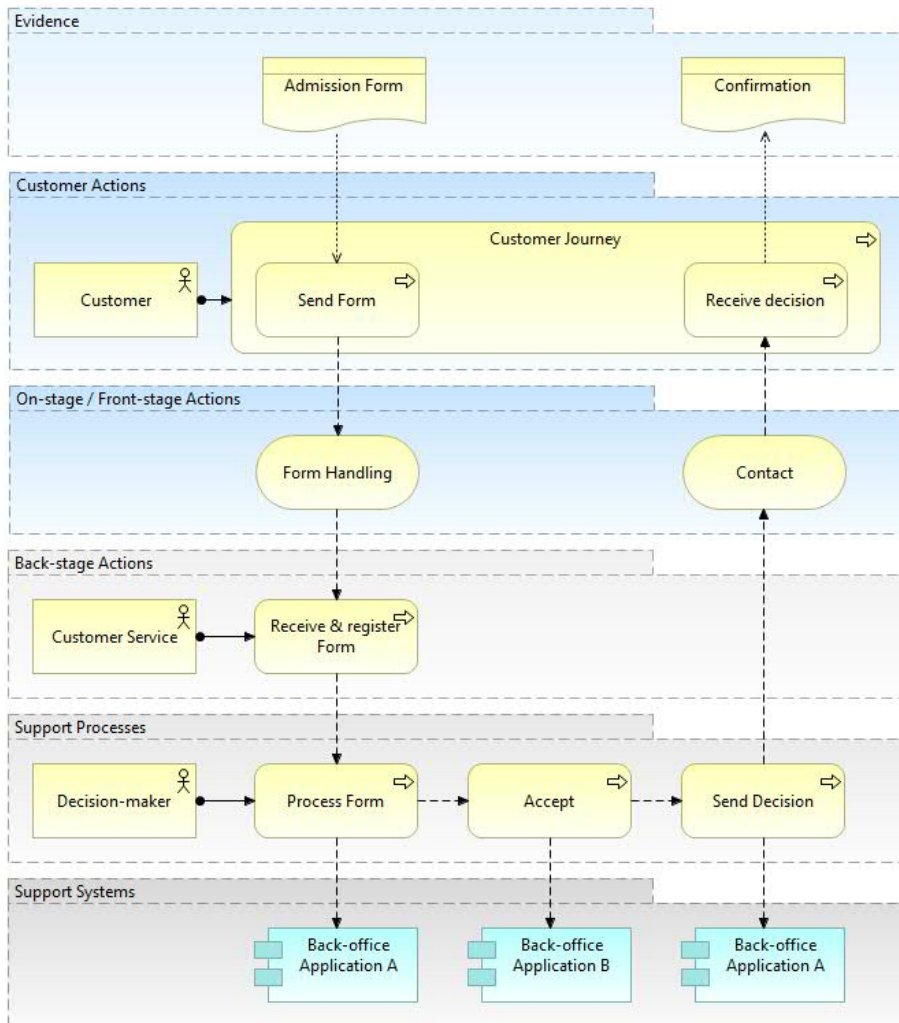


Figure 26: Service Blueprint View 1 (services & flows).

This viewpoint is customer- and service-centric, but it emphasizes also the *"inside-out"* part of the service. With the help of this approach the service-driven development can identify the underlying behavioral and structural impacts of the service that is to be designed. As such, this viewpoint complements the customer-experience driven approach with process- and functional aspects.

There are several variations of this view. This example above focuses on information flows between the layers and elements.

User Story View

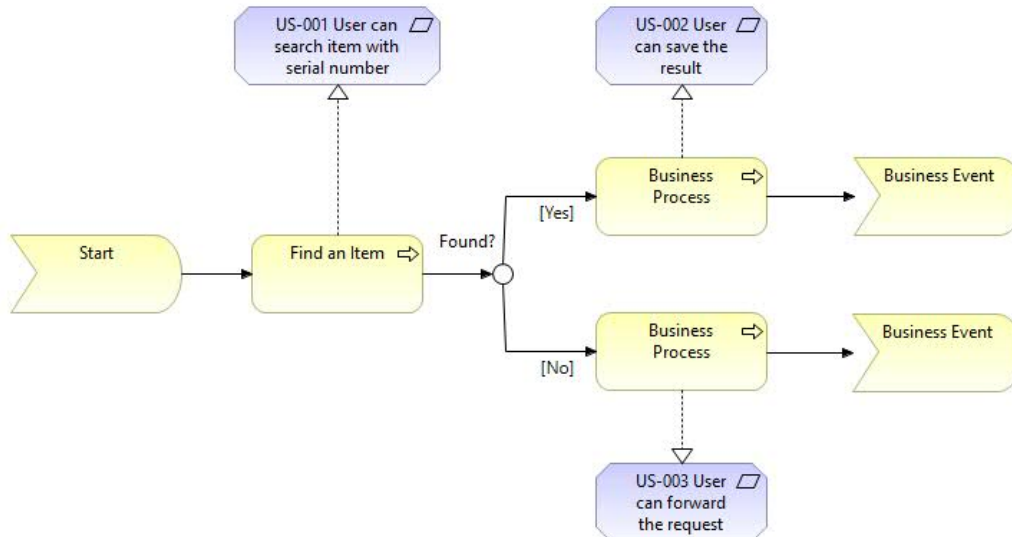


Figure 27: User Story View.

This view can be used to visualization of user stories.

Cloud-Service Models View

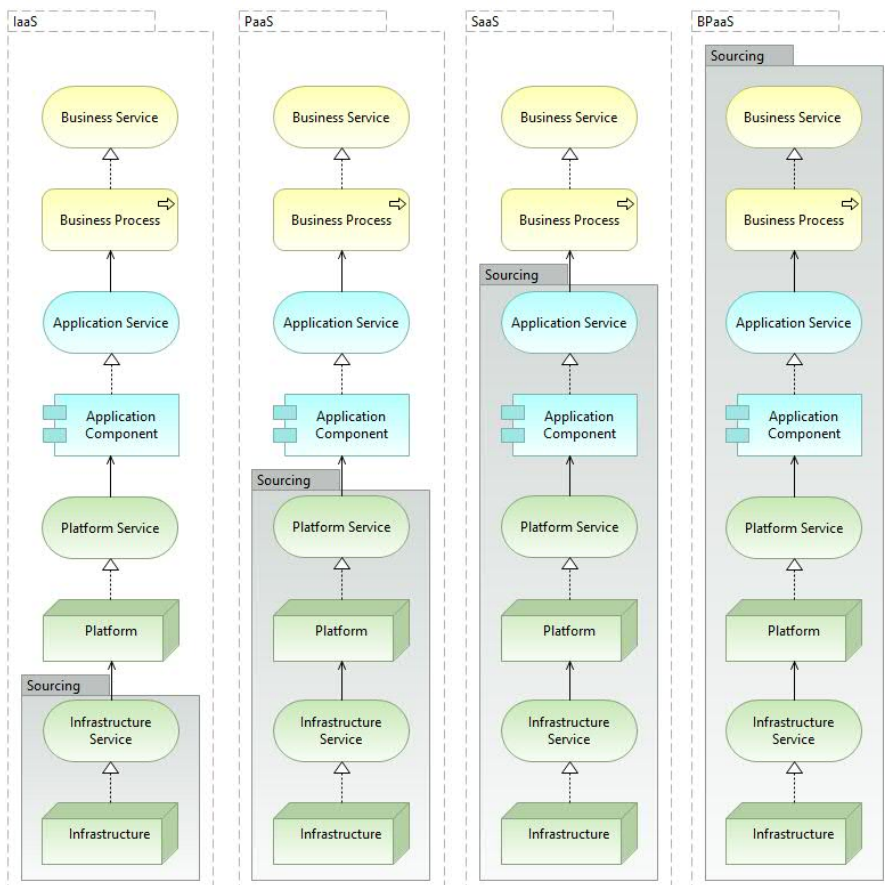


Figure 28: Cloud Service Models View.

Information View

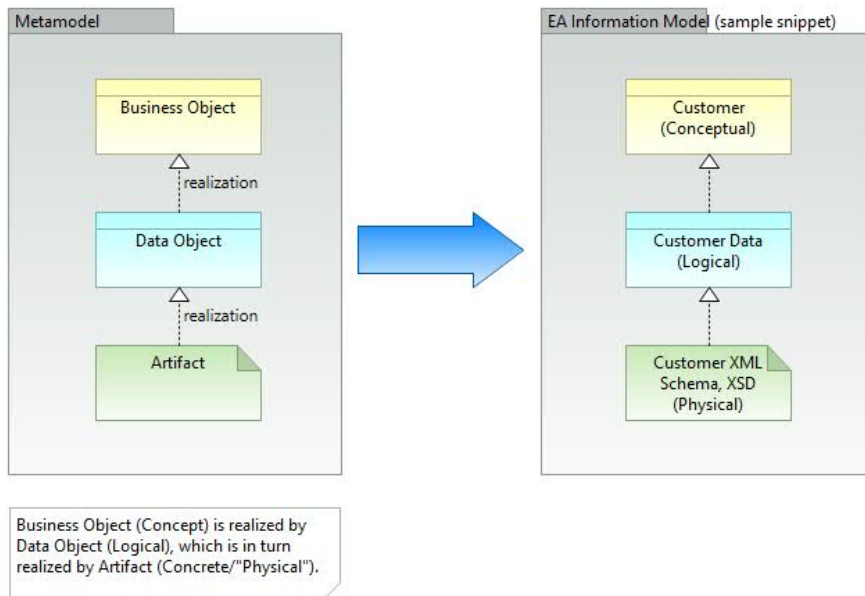


Figure 29: Information View.

Information can be modeled on different abstraction levels as follows: a) conceptual, b) logical and c) physical levels. The diagram above illustrates these abstraction levels.

Conceptual Data Model View

Information architecture of EA contains business objects a.k.a. concepts, that are used in business processes. These concepts and their relations can be represented in a conceptual data model (see figure below).

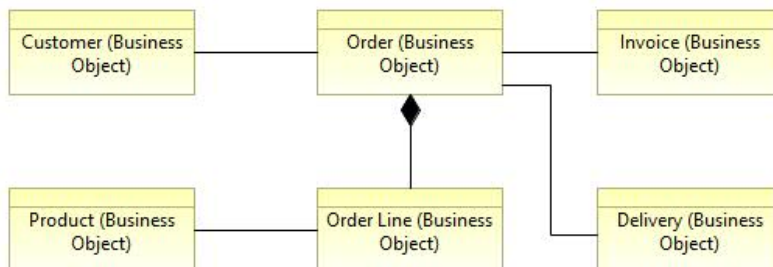


Figure 30: Conceptual Data Model View.

Cardinalities (e.g. one-to-many) can be added on the association relations, depending on the tool used.

"Service" Concept

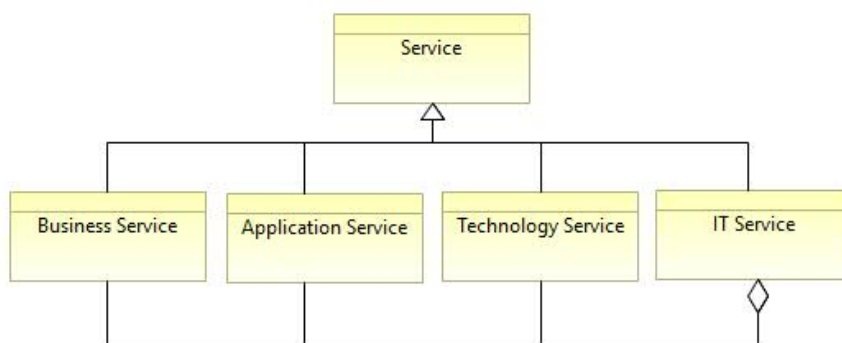


Figure 31: Service Concept.

6. Application Views

Application Architecture Layer Views.

Application Services Map View

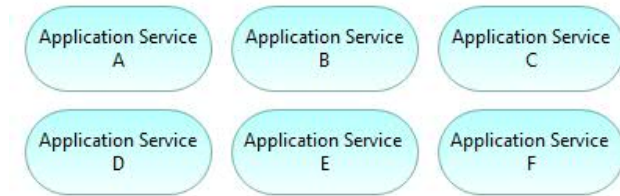


Figure 32: Application Services View.

Applications Map View

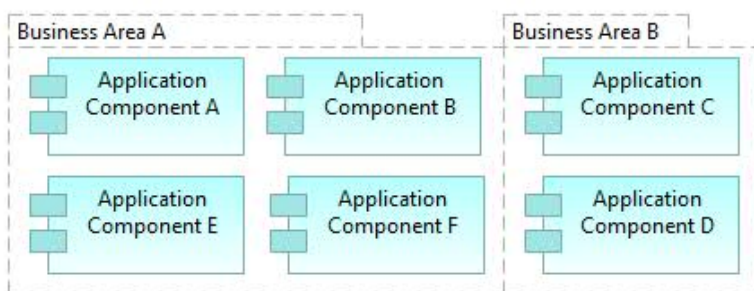


Figure 33: Applications Map View.

Application portfolio, in which applications can be divided into groups e.g. based on business units.

Application Co-Operation View (Data flows)

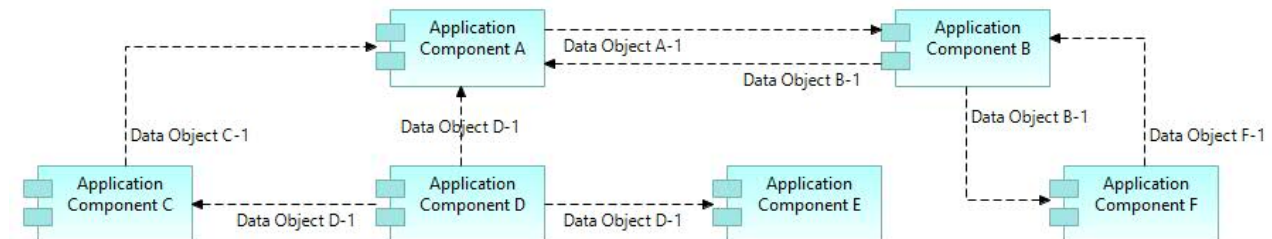


Figure 34: Application Co-operation View.

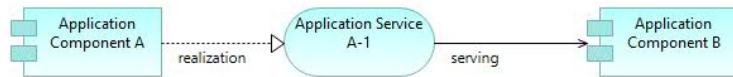
Application Integration View (Dynamic relationships)

Several alternative approaches of modeling data switching between applications are shown in the examples (1 to 9) below.

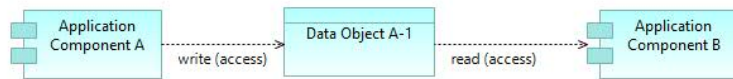
- “Application A” owns a data object “A-1”, which is requested by “Application B”.
- Data flows from “Application A” to “Application B”.
- “Application A” realizes a service “A-1” that is used by “Application B”.
- Practically, “Application B” requests the Application Interface “A-1” and gets response...

Application Integration View

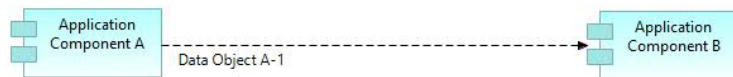
1.
"Application A" provides an application service "A-1" that is used by "Application B".
This illustrates the dependency between the applications.
Structural relationships "Realization" and "Serving" are used.



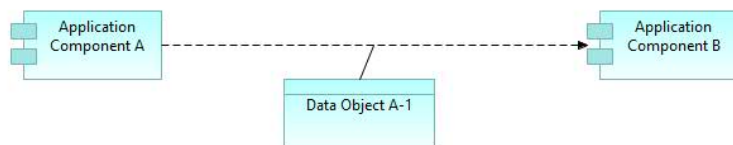
2.
A data object "A-1" flows from "Application A" to "Application B".
The data object is explicitly modeled, and dependency relationship "Access" is used.



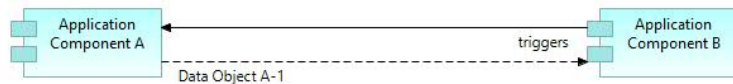
3.
The data object "A-1" flows from "Application A" to "Application B".
The name of the data object is marked as the name (label) of the flow connection between the application components.
Dynamic relationship "Flow" can be used in the cases when there are many data flows in the same diagram - for simplicity and better readability.



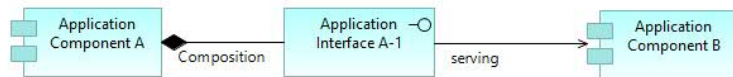
4.
A data object "A-1" flows from "Application A" to "Application B".
The data object is explicitly modeled, and related to "Flow" relation between the applications A and B.
This "relation to relation" is new feature of ArchiMate 3.0.



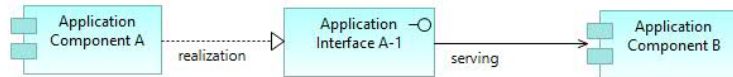
5.
"Application B" is the active part of the communication by initiating the interaction with "Triggering" dynamic relationship.
The data object "A-1" flows from "Application A" to "Application B".
Use of dynamic relationships: "Triggering" and "Flow".



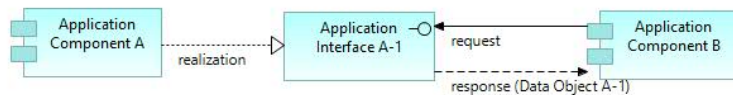
6.
"Application A" has an Interface "A-1" that is serving "Application B".
Use of structural relationship "Composition" and dependency relationship "Serving".
This is in line with ArchiMate specification version 2.x.



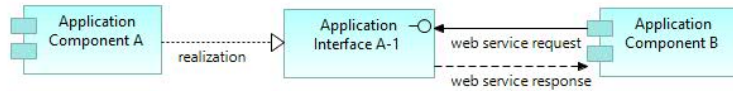
7.
"Application A" realizes an Interface "A-1" that is serving "Application B".
Application Interface "A-1" is structurally part (composed) of "Application A", but the relation can be modeled by using "Realization" relationship according to ArchiMate specification version 3.x.



8.
"Application A" realizes an Interface "A-1" that is requested by "Application B". Interface "A-1" responds to "Application B".
Use of dynamic relationships: "Triggering" and "Flow" together with structural relationship "Realization".
This is in line with ArchiMate specification version 3.x. This can be used when modeling more detailed synchronous integrations (such as request-reply mechanisms).



9.
This is simplification of the previous example, just showing the mechanism / protocol that is used when "Application B" is requesting the Interface "A-1" that is realized by the "Application A".
This is analogous to UML, as reading direction is from the caller application against the Interface / Application that is used.



10.
Application A realizes an Interface "A-1" that is requested by "Application B". Interface "A-1" responds to "Application B".
This approach might be the most informative when using "Application Interface" instead of using "Application Service". Using "Application Interface" is the most suitable to model more detailed dynamics between the applications.
Using "Application Service" can be used, instead, to model high level behavioral dependencies between the applications.
Analogous to approach no 8.
Use of structural relationship "Composition" and dynamic relationships: "Triggering" and "Flow".

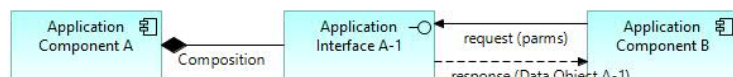


Figure 35: Application Integration View.

Application Structure View

This view is useful in designing or understanding the main structure of applications or components and the associated data; e.g., to break down the structure of the system under construction, to illustrate modularization: what are the sub-systems / components of the system and what interfaces (or services) they provide. Note that application services are the behavioral functionalities that are provided by the structural interfaces (GUIs and/or APIs) - services and interfaces are the "different sides of the same coin".

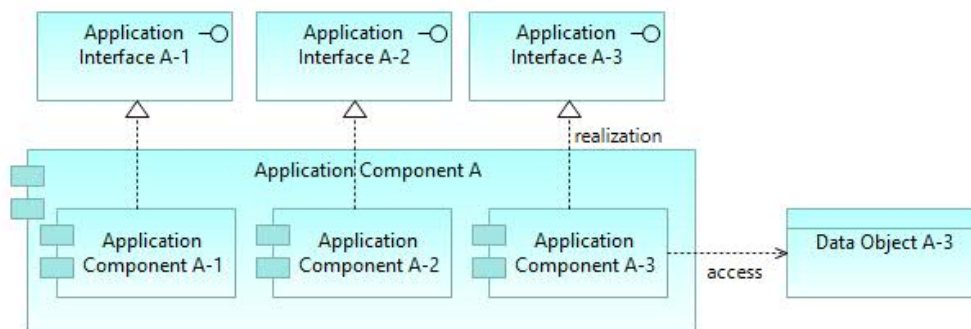


Figure 36: Application Structure View.

Application Functions View

Application functional decomposition: what are the functions the system contains, and which application services they provide?

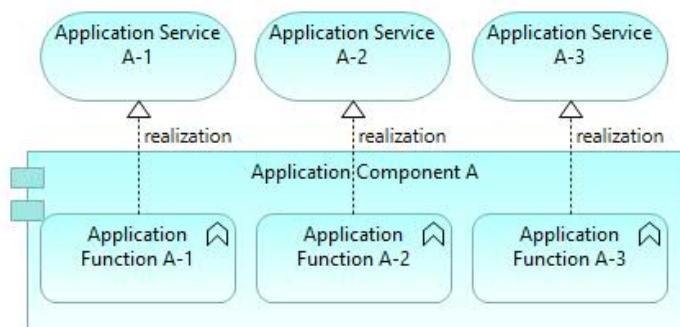


Figure 37: Application Functions View.

Layered View

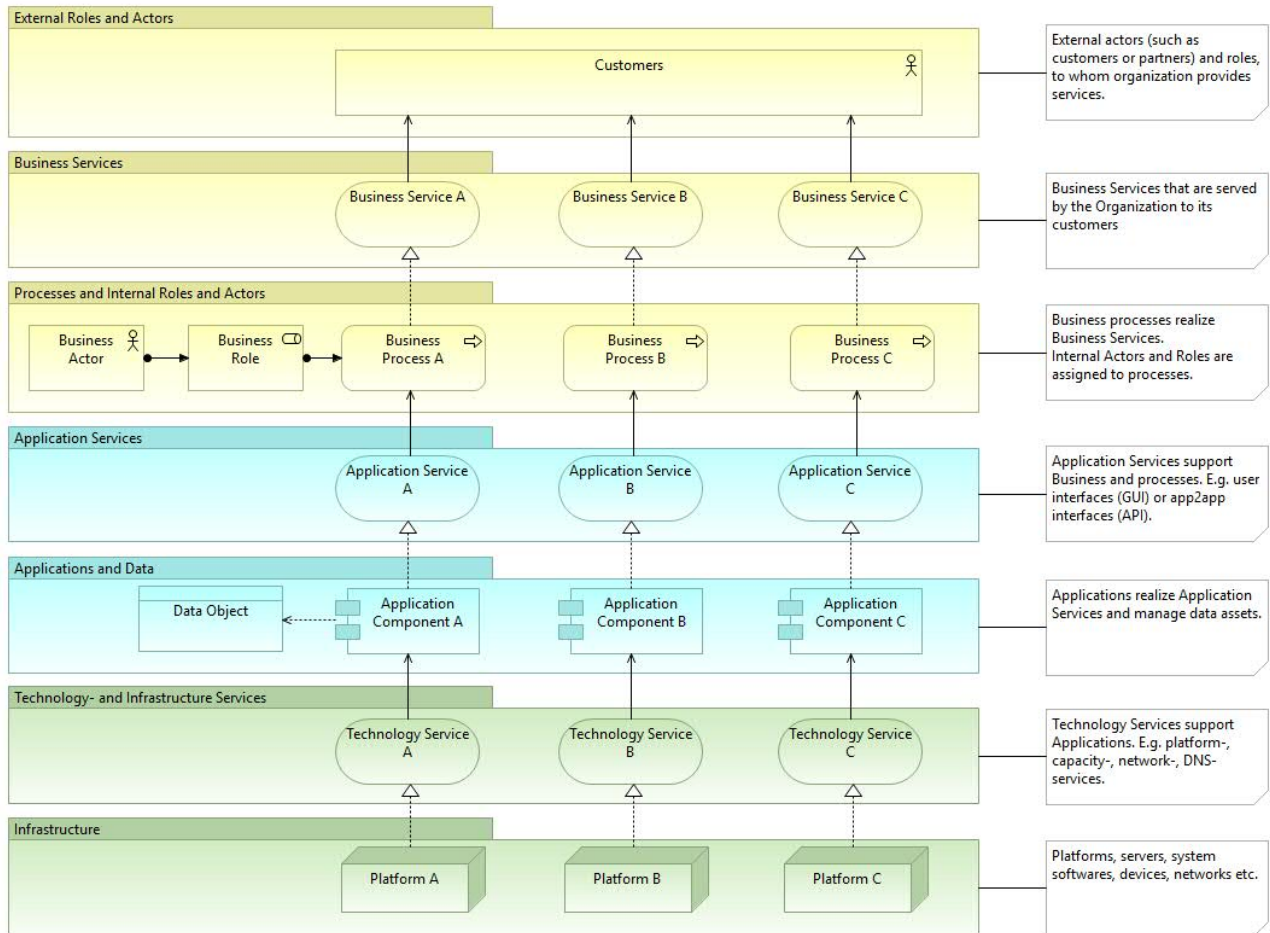


Figure 38: Layered View.

Layered View can be used as an overview context diagram of the target area. The main advantage of this view is to illustrate the usage of applications in business processes and services they provide.

7. Technology Views

Technology Architecture Layer Views.

Infrastructure View

This view represents a platform of an application. This pattern can be used to model a configuration of run-time environment, the deployment of the business application.

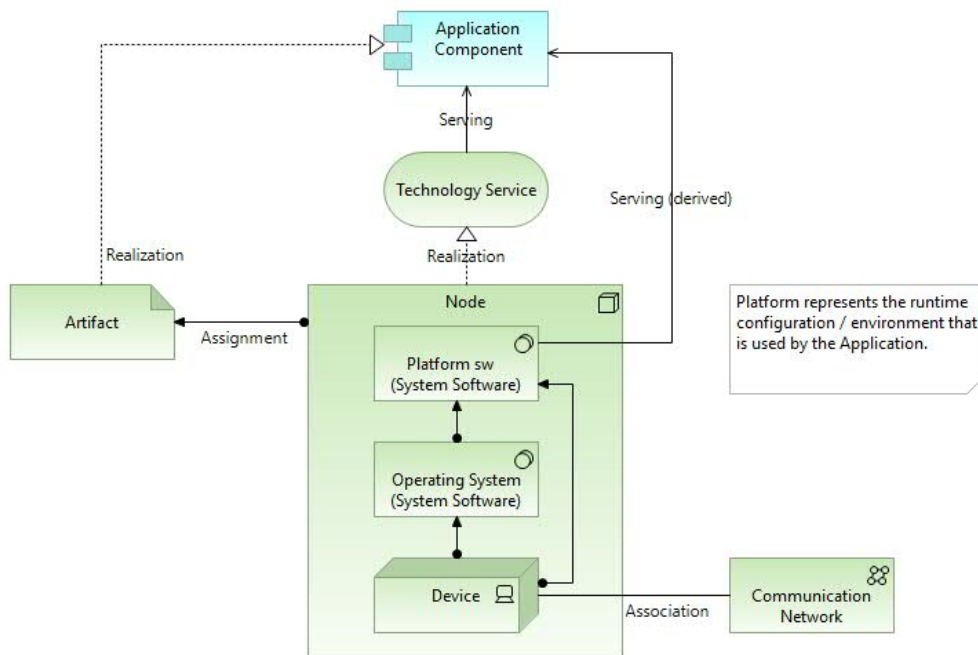


Figure 39: Infrastructure View.

8. Implementation and Migration Layer / Transformation Architecture Layer Views

Generic View

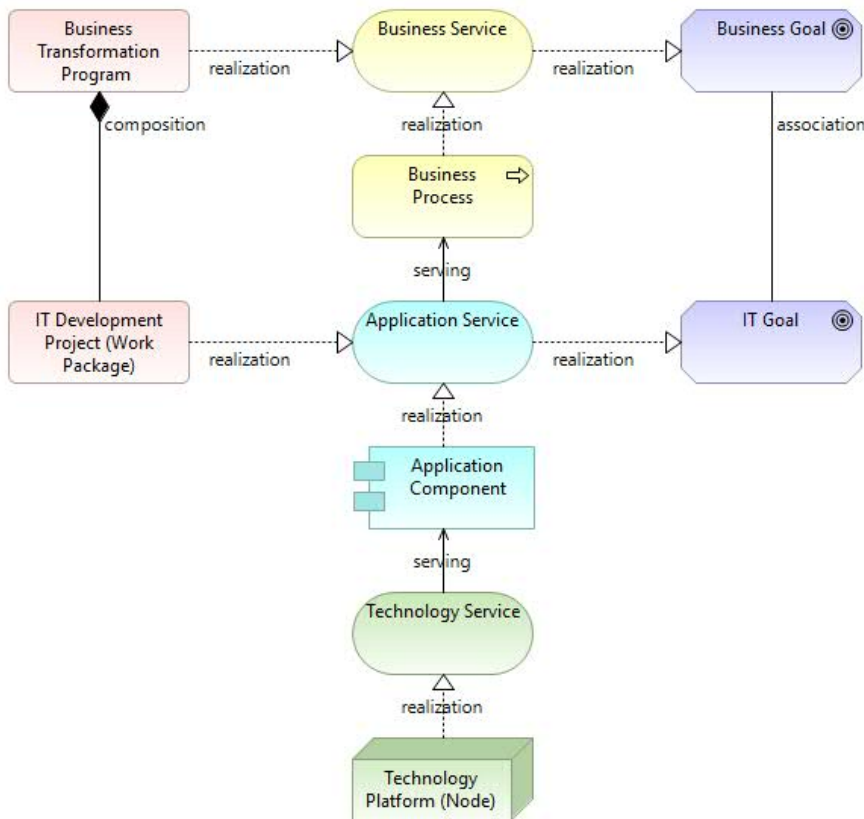


Figure 40: Generic View.

This kind of simplified view can be used e.g. as contextual diagram of specific service, program or project.

Kanban View

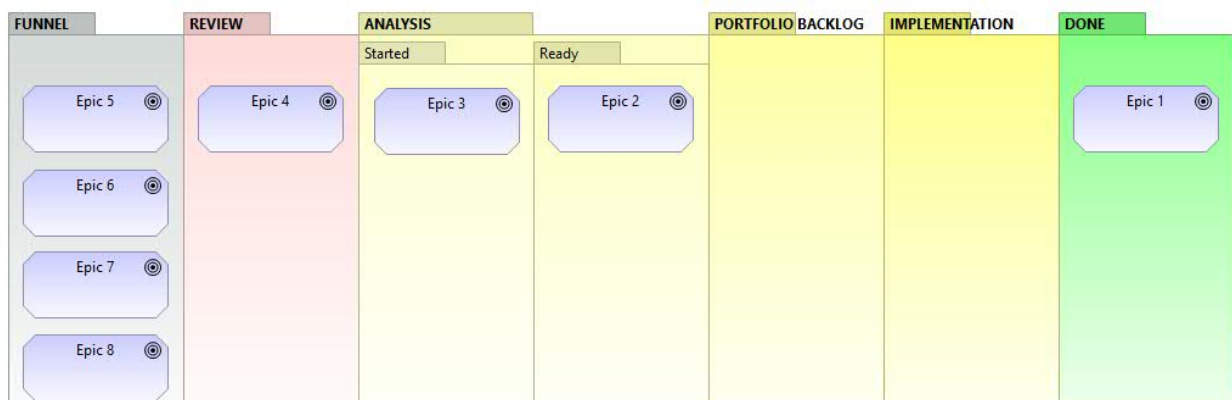


Figure 41: Kanban Board (EA).

Kanban board can be used for visualization of work and the workflow. Kanban board shows how e.g. development requirements, epics, user stories etc. are flowing from backlog to ready state (Done). Kanban board can be applied for diverse purposes depending on the scale and scope of the development case. E.g. in EA level by using "Epics" or in project level by using "User Stories" or "Requirements" as work items. The granularity of work items can vary depending on the case.